SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I P.Harsha Vardhan/192311039 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1

A banking application team is facing frequent failures due to unstructured testing practices. Redesign the testing approach using testing as an engineering activity.

Answer

Introduction

Testing is an engineering activity: it is planned, designed, executed, measured and improved just like any other engineering discipline. Ad-hoc testing (random clicking, no records, no expected results) depends on the tester's memory and mood, so it cannot be repeated, measured or trusted. In a banking application, where an error means wrong money, this is unacceptable. The approach is therefore redesigned as a controlled engineering process.

Introducing systematic test design, tools and standards

- **Systematic test design:** Derive test cases from requirements using black-box techniques (equivalence partitioning, boundary value analysis, decision tables, state transition) and white-box techniques (statement, branch and path coverage). Each test case has an ID, input, precondition, expected result and traceability to a requirement.
- **Tools:** A test management tool (TestRail/Jira Xray) for cases and results, a defect tracker (Jira/Bugzilla), Selenium or Appium for automated regression, JUnit/TestNG for unit tests, JMeter for performance and Jenkins for continuous integration.
- **Standards:** IEEE 829 for test documentation (plan, design, case, log, incident report), IEEE 1044 for defect classification, ISO/IEC 29119 for the test process and templates for entry/exit criteria.

How this improves defect detection and quality

Test cases derived from requirements guarantee coverage, so no feature is silently skipped. Traceability shows which requirement is untested. Recorded results make tests repeatable, so a failure can be reproduced by the developer. Metrics such as defect density and test coverage let the manager decide when quality is acceptable. Automation of regression checks gives fast feedback after every code change.

Techniques to replace ad-hoc testing

- Boundary value analysis and equivalence partitioning for amount and limit fields.
- Decision tables for combinations of rules (balance, daily limit, account status).
- State transition testing for account states (active, dormant, blocked).
- Risk-based testing so that high-risk functions such as fund transfer receive the most effort.
- Reviews and walkthroughs of requirements and test cases before execution.

Practical example

Consider "Fund Transfer" with a rule that a customer may transfer from Rs.1 to Rs.50,000 per transaction. Ad-hoc testers might try only Rs.1,000. The structured design gives equivalence classes (below 1, 1 to 50,000, above 50,000) and boundary values 0, 1, 2, 49,999, 50,000 and 50,001. A decision table adds insufficient balance and blocked account cases. When the boundary value 50,000 was rejected by a "less than" bug, only the structured set detected it, whereas random tests never reached it.

Conclusion

Treating testing as an engineering activity, with defined design techniques, tools and standards, converts unpredictable testing into a measurable and repeatable process that detects more defects earlier and raises the quality of the banking application.

Question 2

An ATM system is producing inconsistent results due to lack of a defined testing process. Implement a structured testing process across SDLC phases and apply it to ATM withdrawal.

Answer

Introduction

A defined testing process makes results consistent because every release passes through the same planned stages. Testing is integrated with every SDLC phase (the V-model idea), so that verification and validation are carried out continuously instead of only at the end.

Stages of the structured process and their relevance

SDLC phase	Testing activity	Relevance to quality
Requirements	Requirement review, testability check, test planning	Removes ambiguous or missing requirements before they become defects
Design	Review of design, design of test cases, test environment planning	Ensures every design element is testable and covered
Coding	Unit testing, code review, static analysis	Finds defects at the cheapest point to fix
Integration	Integration testing of card reader, keypad, bank server, cash dispenser	Detects interface and data-flow defects
System / Acceptance	System, performance, security and user acceptance testing	Confirms the complete ATM meets user needs
Maintenance	Regression testing and defect analysis	Protects working features after every change

Application to ATM withdrawal

- **Requirement stage:** Clarify rules such as minimum Rs.100, multiples of Rs.100, daily limit Rs.20,000, PIN attempts limited to three, sufficient balance and sufficient cash in the machine.
- **Test design:** Create cases for valid withdrawal, amount not a multiple of 100, amount above limit, insufficient balance, wrong PIN three times, card captured, cash cassette empty and network failure during transaction.
- **Unit and integration:** Test the balance-check function and the message exchange between ATM and bank server; verify that the account is debited only if cash is dispensed.
- **System:** Execute all cases on the integrated ATM; include security (PIN masking) and performance (response time under 5 seconds).
- **Acceptance and regression:** Customer scenarios are verified; any fix is followed by a regression suite.

How process consistency reduces defects

When the same entry and exit criteria, templates and test cases are used for every release, results become comparable and reproducible, so an inconsistency indicates a real defect rather than a testing difference. Early reviews prevent defects, defect logs reveal weak areas, and regression testing prevents the same fault from returning.

Expected outcomes

- Consistent and repeatable ATM test results.

- Fewer failures such as double debit or wrong cash dispensed.
- Early defect detection and lower cost of correction.
- Measurable quality through coverage and defect metrics, and higher customer confidence.

Question 3

A web application shows a mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions (error, defect, failure).

Answer

Basic definitions

- **Error (mistake):** A human action that produces an incorrect result, for example a developer misunderstanding a requirement.
- **Defect (fault, bug):** An imperfection in the code or document that results from an error; it is the cause present in the software.
- **Failure:** The observable deviation of the system from expected behaviour when the defect is executed.

Identifying them in the scenario

Item	Login	Transaction
Error	Developer assumes user ID is case-insensitive although the requirement says it is case-sensitive	Developer forgets that the amount may contain decimals
Defect	Condition in code converts the user ID to lower case before comparison	Variable storing the amount is declared as integer, truncating Rs.100.75 to Rs.100
Failure	"user01" logs in with the password of "USER01"	Rs.100.75 is deducted as Rs.100 and the balance is wrong

Relationship

The relationship is a chain: Error leads to Defect, and Defect leads to Failure. Not every defect causes a failure: a defect in a code path never executed, or masked by particular input, stays hidden. Conversely, a failure may also come from the environment or wrong test data, so a failure must be analysed before a defect is declared.

Effect of improper understanding

- Testers report symptoms as causes, so developers search in the wrong place.
- Duplicate and invalid defect reports increase and waste effort.
- Severity is misjudged; a serious defect may be treated as a minor error.
- Metrics such as defect density become unreliable and process improvement targets the wrong activity.

Improving defect reporting clarity

- **Use a standard template:** ID, title, module, environment, steps to reproduce, expected result, actual result, severity, priority, status and attachments.
- Describe the failure observed (actual versus expected) and, once found, the defect location and probable error origin.
- One defect per report, with a precise title such as "Transaction amount 100.75 stored as 100".

- Attach screenshots and logs, and review reports before assignment.

Conclusion

Clear use of the terms error, defect and failure makes root-cause analysis faster and defect reports unambiguous, improving communication between testers and developers.

Question 4

Most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address the issue.

Answer

Introduction

The seven testing principles guide where and how testing effort should be spent. The observed situation is a direct example of defect clustering.

Principles and their effect on test planning

- **Defect clustering:** A small number of modules usually contain most defects (the Pareto rule: about 80% of defects in 20% of modules). Test planning should allocate more time, more senior testers and deeper techniques to these modules.
- Testing shows presence of defects, not their absence: Passing tests do not prove the product is defect-free, so risk-based stopping criteria are defined.
- **Exhaustive testing is impossible:** Because all inputs cannot be tested, test cases are selected by risk and technique.
- **Early testing:** Reviewing requirements and design of the weak modules prevents defects.
- **Pesticide paradox:** The same tests stop finding defects; test cases must be reviewed and updated regularly, especially in clustered modules.
- **Testing is context dependent:** Safety or banking software needs more rigorous testing than a brochure website.
- **Absence-of-errors fallacy:** A defect-free product that does not meet user needs is still a failure.

Prioritizing test cases

- Collect defect data from earlier releases and rank modules by defect density.
- Rate each requirement by business impact and likelihood of failure (risk exposure = impact x likelihood).
- Run high-risk and high-defect-module tests first (smoke and critical-path tests), then medium and low risk.
- Add more boundary, negative and integration cases to clustered modules, and keep regression tests for the stable modules minimal and automated.

Improvement in testing efficiency

Effort is directed where defects are likely, so more defects are found per hour of testing, the critical defects are found earlier, retesting is controlled and the schedule delay is reduced.

Real-time example

In an online shopping application, defect analysis showed that the Payment and Cart modules produced 70% of all defects while Catalogue and Help produced very few. The plan allotted 60% of testing effort to Payment and Cart, added boundary and failure-recovery tests (payment gateway timeout, coupon combinations), automated the stable Catalogue regression and rotated test data. Critical defects were discovered in the first test cycle and the release was no longer delayed.

Question 5

A company maintains a defect repository but does not use it effectively, resulting in repeated defects. Apply defect classification and repository concepts.

Answer

How defects originate

Defects are injected in every phase of development: requirement defects (missing or ambiguous requirements), design defects (wrong algorithm, poor interface), coding defects (logic and syntax mistakes) and testing/environment defects (wrong test data or configuration). The causes are human error, time pressure, poor communication and changing requirements.

How defects are categorized

Basis	Classes
Origin / phase	Requirements, design, coding, testing, documentation
Type	Functional, logic, interface, data, computation, performance, security, usability
Severity	Critical, major, minor, trivial
Priority	High, medium, low
Status	New, assigned, fixed, retested, closed, reopened

Using repository data for test design and prevention

- Analyse recurring defect types, for example boundary errors, and add boundary tests for all new modules.
- Identify modules with high defect density and test them more deeply.
- Reuse past defects as checklists for reviews and as regression cases.
- Perform root-cause analysis, then change the process (training, coding standards, checklists) so that the cause does not recur.

Improvements to avoid recurring defects

- Make a defect log mandatory with complete fields and standard classification (IEEE 1044).
- Hold a periodic defect review meeting and a retrospective at the end of each release.
- Link each defect to its requirement and test case; add a regression test for every fixed defect.
- **Use dashboards to show trends:** defects by module, type, phase and age.

Expected quality improvements

Fewer repeated defects across releases, early defect removal, targeted test design, shorter fixing time, improved predictability of release quality and a measurable decline in defect density.

Question 6

A mobile banking application frequently crashes during fund transfer. Apply software testing principles to identify the root cause and prevent failures.

Answer

Identifying the root cause

- **Reproduce the failure:** Record the steps, device, OS version, network and data used, and check logs and crash reports.

- **Isolate the conditions:** Vary one factor at a time (amount, beneficiary, network speed, device memory) to find the trigger.
- **Apply defect clustering:** Fund transfer is a high-risk, defect-prone module, so inspect its code and interfaces first.
- **Trace the chain:** error (developer misses a timeout case), defect (null response from the server not handled) and failure (application crash).
- **Likely root causes:** Unhandled exceptions, memory leaks, no timeout handling, concurrent transactions, invalid input and incompatibility with some devices.

How systematic testing reduces such issues

Systematic testing covers normal, boundary and negative cases deliberately. It includes unit tests of the transfer logic, integration tests with the bank API, stress tests with many users, network interruption tests and compatibility tests on a matrix of devices. Because each case is documented and repeatable, defects are found before release and fixes are verified by regression tests.

Illustration

Test scenario	Input / condition	Expected result
Boundary amount	Rs.1 and the maximum limit	Transfer accepted
Above limit	Limit plus Rs.1	Clear error message, no crash
Network drop	Connection lost after confirm	Transaction status checked, no duplicate debit
Double tap	Submit pressed twice	Only one transaction
Low memory device	Transfer on an older phone	App continues to run

Improvements in testing practices

- Early testing and review of the transfer design and error handling.
- Automated regression suite run on every build (CI).
- Device cloud testing (BrowserStack/Firebase Test Lab) and performance testing with JMeter.
- Crash analytics tools to learn from production issues.
- Root-cause analysis and defect trend review after every release.

Question 7

A development team confuses debugging with testing. Differentiate both processes and suggest a structured approach to integrate them.

Answer

Concepts

Testing is the process of executing software with the intention of finding defects and judging whether it meets its requirements. Debugging is the process of locating the cause of a known failure, correcting the defect and verifying the correction.

Differences

Aspect	Testing	Debugging
Purpose	Detect failures and show conformance	Find and remove the cause of a failure
Performed by	Testers (also developers)	Developers

Aspect	Testing	Debugging
Input	Test cases and requirements	Defect report and failing test
Planned	Planned, designed and can be scheduled	Depends on defects found
Outcome	Defect reports, test results	Corrected code
Automation	Largely automatable	Mostly manual, with tool support

Roles in software quality

Testing measures quality and gives visibility of defects; debugging improves quality by removing them. Neither can replace the other: testing without debugging leaves defects, and debugging without testing never shows what to fix or whether the fix is correct.

Effect of improper usage

- If developers "test" by debugging only, they test only what they expect, so many defects remain undetected.
- Testers spending time on locating causes delay further testing.
- Defects are fixed without reports, so there is no history or retesting, and regressions occur.
- Quality metrics become unreliable.

Structured approach to integrate them

- **Step 1:** Tester designs and executes tests and logs a failure with steps to reproduce.
- **Step 2:** Developer reproduces the failure and debugs: locates the fault, using logs, breakpoints and backtracking.
- **Step 3:** Developer fixes the code and performs unit testing.
- **Step 4:** Tester performs confirmation (re-)testing of the fix.
- **Step 5:** Regression testing checks that nothing else broke.
- **Step 6:** Defect is closed; the cause is recorded for process improvement.

Question 8

An e-commerce platform shows inconsistent behavior across browsers. Design a cross-browser testing strategy.

Answer

Why defects originate

- Different rendering engines (Blink, Gecko, WebKit) interpret HTML, CSS and JavaScript differently.
- Use of features not supported by older browser versions.
- Different screen sizes, resolutions, operating systems and default fonts.
- Differences in cookie, caching and security settings, and in third-party plug-ins and payment scripts.

Strategy for cross-browser testing

- **Define scope:** Use analytics to choose the browsers, versions, devices and operating systems used by customers (for example Chrome, Edge, Firefox, Safari on Windows, macOS, Android, iOS).
- **Prioritise by risk:** Test critical flows first (search, cart, checkout, payment, login), then secondary pages.

- Design test cases once and execute on the compatibility matrix: layout, functionality, navigation, forms, performance and security.
- **Execute:** Manual exploratory checks for visual issues and automation for repeatable functional flows.
- **Report and retest:** Log browser and version in each defect; run regression after fixes.

Illustration

Example: the "Place order" button works in Chrome but does nothing in Safari because of unsupported JavaScript syntax. A compatibility matrix with the checkout test case executed on each browser exposes the failure before release. The fix is a polyfill, and the retest on all browsers confirms it.

How systematic testing improves reliability

A defined matrix and repeatable test cases give predictable coverage of the environments that customers actually use, so defects are found before release, and regression runs ensure that fixes for one browser do not break another.

Tools and techniques

- **Tools:** Selenium Grid/WebDriver, BrowserStack, LambdaTest, Sauce Labs, Cypress, Playwright and browser developer tools.
- **Techniques:** Compatibility matrix and pairwise testing (to limit combinations), responsive design testing, visual regression comparison, HTML/CSS validation and progressive enhancement.

Question 9

A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques.

Answer

Categorizing the defects

Classification basis	Class for payroll defects	Example
Origin	Coding / design	Wrong formula for net salary
Type	Logic and computation defect	Net = Gross + Deductions instead of Gross - Deductions
Severity	Critical	Employees are paid wrongly
Priority	High	Must be fixed before salary run
Detection phase	System testing	Found after integration of tax module

Further examples of logic defects: wrong relational operator in the tax slab condition (> instead of >=), incorrect loop for overtime hours, wrong order of operations, missing rounding rule and data type truncation.

How understanding defect classes improves testing

- Testers know which defect types are likely (computation and boundary defects) and design targeted tests.
- Severity and priority guide the order of testing and fixing.
- Classification of origin shows which phase needs better review.
- Metrics by class show trends and help decide where to apply more effort.

Illustration

For a tax rule "income up to Rs.5,00,000 is exempt", boundary tests with Rs.4,99,999, Rs.5,00,000 and Rs.5,00,001 reveal the incorrect operator. Basis path and decision-table tests verify combinations of allowances and deductions.

Prevention in future releases

- Code reviews and inspection against a checklist of logical errors.
- Unit tests for every formula with expected values verified by finance staff.
- Static analysis tools and coding standards.
- Regression tests for every defect found, and parallel runs against the old payroll results.

Improvements in testing strategy

Use boundary value analysis, decision tables and white-box coverage (branch and condition) for calculation modules; automate regression; include payroll experts in requirement reviews and acceptance testing; and update tests whenever tax or policy rules change.

Question 10

A software team skips proper test case design and directly executes random tests. Redesign the testing process.

Answer

Redesigned process

The testing activity is organised as an engineering process: Planning, then Test analysis and design, then Implementation of test cases and data, then Execution, then Evaluation of exit criteria and reporting, and finally Closure. Each step produces documents that can be reviewed and reused.

Importance of structured test case design

- **Coverage:** Each requirement is linked to test cases, so nothing is missed (traceability).
- **Repeatability:** Anyone can execute the same test and obtain comparable results.
- **Efficiency:** A small, well-chosen set of tests finds more defects than a large random set.
- **Early defect detection:** Designing tests exposes errors in requirements.
- **Reuse and measurement:** Cases serve regression and provide coverage metrics.

Illustration

Take a field "Age" that accepts 18 to 60. Random testing might use 25 and 30, and would pass. Structured design gives the partitions (below 18, 18 to 60, above 60) and boundary values 17, 18, 19, 59, 60, 61. If the code uses "age > 18", the value 18 fails, and a defect is found with only six tests.

Suitable techniques

- **Black box:** equivalence partitioning, boundary value analysis, decision tables, state transition and use case testing.
- **White box:** statement, decision and condition coverage, and basis path testing.
- **Experience based:** Error guessing and exploratory testing (time-boxed and documented) to supplement designed tests.
- Review and traceability matrix to confirm coverage; tools such as TestRail for management.

Conclusion

Structured test case design makes testing systematic and measurable and increases the number and criticality of defects detected for the same effort.

Question 11

A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue.

Answer

Analysis of the issue

Data inconsistency means the same data differs between modules, for example a patient's allergy recorded in the Registration module but missing in the Pharmacy module. In healthcare this can threaten patient safety, so risk is very high and the principle "testing is context dependent" demands rigorous testing. Defect clustering suggests examining the interfaces between modules first.

How defects originate during development

- **Requirement defects:** Data definitions, formats and ownership not clearly specified.
- **Design defects:** Duplicate tables, no single source of truth, wrong data mapping.
- **Coding defects:** Different data types or lengths, failure to update all dependent tables, unhandled transaction rollback.
- **Integration defects:** Interface or API mismatches and timing (synchronisation) problems.
- **Change defects:** Changes in one module without impact analysis.

How systematic testing ensures data integrity

- Integration testing of data flow between modules (Registration, Billing, Pharmacy, Lab).
- **Database testing:** constraints, referential integrity and CRUD operations verified against the database.
- **Transaction tests:** atomicity, rollback and concurrent update scenarios (ACID).
- Boundary and negative data tests; data migration and reconciliation testing.
- **End-to-end scenarios, for example:** register patient, add allergy, prescribe medicine, then confirm the same data in every module.
- Regression tests after every change.

Improvements in defect tracking and reporting

- Use a defect tracker with module, data involved, steps, expected and actual results and severity (patient-safety defects are critical).
- Record the origin phase of each defect and perform root-cause analysis.
- Use dashboards for defect trends, and give traceability from defect to requirement and test case.
- Hold daily triage meetings so that data defects are assigned and retested quickly.

Question 12

A company faces repeated defects in its software releases despite testing efforts. Apply the concept of the defect repository.

Answer

Analysis of the problem

The repeated defects show that the knowledge gained from earlier defects is not used: fixes are not covered by regression tests, root causes are not removed and tests are not updated (pesticide paradox). A defect repository is a central, structured database of all defects with their attributes and history, and it is the source for this analysis.

How repository data supports test design

- Lists the defect types and modules that fail most, so test cases are added there.
- Each past defect becomes a regression test case and review checklist item.
- Shows which requirements and conditions were missed, for better equivalence and boundary design.
- Gives data to estimate effort and risk for the next release.

Identifying defect patterns

Analysis	Method	Pattern revealed
By module	Pareto chart of defect count	20% of modules contain 80% of defects
By type	Group by defect class	Repeated boundary or null-handling defects
By phase	Origin versus detection phase	Many requirement defects detected late
By release	Trend / reopen rate	Same defect reappearing after fixes
By cause	Root cause (fishbone, 5 whys)	Missing code review or unclear requirement

Improvements to prevent recurring defects

- Require complete, classified defect records with root cause.
- Add a regression test for each fixed defect and update test suites every release.
- Review repository trends in release retrospectives and act on the causes (training, checklists, standards).
- **Improve the earlier phases:** reviews of requirements and design.
- Search the repository for similar defects when a new one is logged.

Question 13

A web-based learning system fails during peak usage due to untested scenarios. Apply testing principles to identify gaps.

Answer

Gaps identified

- Only functional testing with a single user; no performance, load, stress or scalability testing (violates "testing is context dependent").
- Peak scenarios (examination or registration time) were not identified in test planning.
- Test environment smaller than production; unrealistic data volume.
- No monitoring of response time, memory, CPU and database connections.

- Incomplete risk analysis and exit criteria.

Effect of test planning and coverage on performance

A test plan that includes performance objectives (for example 5,000 concurrent users with response time under 3 seconds), workload models, environments and coverage of critical user flows (login, content access, quiz submission) reveals bottlenecks such as database locking, thread exhaustion and memory leaks before release. Low coverage leaves such scenarios unknown, so they appear for the first time in production.

How better strategies improve reliability

- Load testing verifies behaviour at expected peak load.
- Stress testing finds the breaking point and checks graceful recovery.
- Endurance (soak) testing finds memory leaks.
- Spike testing checks sudden increases of users.
- Results lead to tuning (caching, indexing, load balancing) and the system becomes predictable and available.

Illustration

Example: in a simulated online examination, 3,000 students submit answers within the same minute. JMeter shows the response time rising to 20 seconds and the database connection pool being exhausted at 1,500 users. After the pool size and queries are tuned, a retest meets the target of 3 seconds at 3,000 users.

Enhancements

- Include non-functional requirements and performance tests in the plan.
- Use production-like environments and data; JMeter, LoadRunner or Gatling.
- Run performance tests in every release and monitor production (APM tools).
- Define clear exit criteria and capacity planning.

Question 14

Testers are not involved in early development stages. Apply the role of the tester in the SDLC to improve the situation.

Answer

Role of the tester in the SDLC

Phase	Tester's role
Requirements	Review for completeness, clarity and testability; identify risks; draft test plan
Design	Review design; design test cases and test data; plan environment
Coding	Support unit and integration testing, static analysis and reviews
Testing	Execute tests, log defects, retest and regression test
Deployment / Maintenance	Smoke test, acceptance support, regression for changes

Impact of early involvement on quality

- Defects found in requirements and design cost far less to fix than defects found after release.

- Test cases are ready earlier and aligned with the requirements, which shortens the testing phase.
- Fewer misunderstandings between developers, testers and customers.
- Risks are known early, so the schedule is more accurate.

Practical example

In a library management system the requirement says "a member can borrow books". A tester involved at the requirement stage asks how many books, for how many days and what happens with fines. The missing rules are added before coding. Without the tester, the developer would have assumed rules and the defect would have been found in system testing, requiring rework of the design, code and tests.

Changes in team collaboration practices

- Include testers in requirement, design and code review meetings and in sprint planning.
- **Use Agile practices:** daily stand-ups, three-amigos sessions (business, developer, tester) and a shared definition of done.
- Adopt shift-left testing and test-driven or behaviour-driven development.
- Use a shared defect tracker, shared documentation and continuous integration for fast feedback.
- Promote mutual respect and a "quality is everyone's responsibility" culture.

Question 15

A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps.

Answer

Identifying the gaps

Passing tests only show that those tests passed (testing shows presence of defects, not absence). Since exhaustive testing is impossible, the failures arise from the inputs and paths not selected. Gaps are located by tracing requirements to test cases (traceability matrix), measuring code coverage with tools (JaCoCo, Coverage.py), reviewing the equivalence classes and boundaries and analysing the defects found after release.

How proper coverage ensures reliability

Coverage measures how much of the requirements, input domain and code has been exercised. High, risk-based coverage means that most functions, paths and data conditions have been verified, so the chance that a user meets an untested situation is small and the confidence in the system is greater. Coverage is a measure, not a guarantee, so the quality of tests also matters.

Improving test case design

- **Requirement coverage:** Every requirement has at least one positive and one negative test.
- **Input coverage:** Equivalence partitioning and boundary value analysis for each field.
- **Logic coverage:** Decision tables and cause-effect for combined conditions.
- **Code coverage:** Statement, branch and condition coverage targets (for example 100% statement and 90% branch for critical modules).
- Review test cases and update them regularly to avoid the pesticide paradox.

Illustration

A discount function gives 10% for orders above Rs.1,000. Tests with Rs.500 and Rs.2,000 pass. Coverage shows that the branch for exactly Rs.1,000 and for negative values was never run. Adding Rs.1,000, Rs.1,001 and -1 reveals that Rs.1,000 receives the discount incorrectly.

Techniques for better validation

- Black-box and white-box techniques together, and pairwise testing for combinations.
- Error guessing and exploratory testing.
- Peer reviews of test cases and requirements traceability.
- Mutation testing to check the strength of tests, and regression automation.

Question 16

A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting.

Answer

Importance of clear definitions

Using the terms error, defect and failure precisely, together with agreed values for severity, priority and status, lets everyone understand the same report in the same way. A tester reports the failure observed; the developer finds the defect; the team studies the error that caused it. Clear definitions also allow defects to be counted and tracked consistently across modules and releases, so duplicates are recognised and the status of every defect is known.

How structured reports enhance communication

A structured report contains all the information needed to reproduce and fix the defect, so developers do not need to ask questions. It reduces delay, avoids misunderstanding between teams in different locations and gives managers reliable data.

Field	Example (logistics application)
Defect ID / Title	LG-045: Shipment status not updated after delivery scan
Module / Version	Tracking / Release 2.3
Environment	Android 14, Chrome 126, test server
Steps to reproduce	1. Login as driver 2. Scan package barcode 3. Select "Delivered"
Expected result	Status changes to Delivered and customer is notified
Actual result	Status remains "In transit"; no notification
Severity / Priority	Major / High
Attachments / Status	Screenshot, log file / New

Improvements in reporting standards

- Adopt a standard template and defect life cycle (new, assigned, fixed, retested, closed or reopened) in a tool such as Jira.
- Follow IEEE 829 / IEEE 1044 for incident reports and classification.
- One defect per report, with a precise title and reproducible steps; check for duplicates first.
- Define severity and priority guidelines and the response time for each.
- Review reports daily in triage meetings and train testers in report writing.
- Track metrics such as detection time and reopen rate for improvement.

Question 17

A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches.

Answer

Process-based approach

A process-based approach defines a common, documented test process (for example in line with ISO/IEC 29119 or TMMi) that all teams follow: planning, design, execution, reporting and closure, with standard templates, entry and exit criteria, roles and metrics.

How standardised processes improve quality

- Same quality criteria and techniques for every module, so results are comparable.
- Fewer omissions, as checklists and templates remind teams of required activities.
- Reusable test assets reduce effort and errors.
- Metrics collected in the same way make quality visible and improvements possible.
- New team members become productive quickly, and dependence on individuals decreases.

Illustration

Example: In an HR system, the Payroll team uses boundary value analysis and records results in a tool, while the Leave module team tests informally. Management introduces one test process: a common test plan template, a requirement-to-test traceability matrix, entry criteria (build is stable and smoke test passed) and exit criteria (all critical tests passed, no open critical defects, 90% requirement coverage). After adoption, defects found in the Leave module increased in system testing and decreased in production.

Methods to ensure consistency

- Create a test policy, test strategy and templates (plan, case, report, defect).
- Provide training and a common tool set (test management, defect tracking, automation framework).
- Conduct reviews and audits by a quality assurance group; use checklists.
- Define metrics (coverage, defect density, defect removal efficiency) and review them regularly.
- Continuous process improvement through retrospectives and lessons learned.

Question 18

A system fails after deployment due to untested edge cases. Apply testing principles and explain boundary testing.

Answer

Why such cases were missed

- Time and resources were limited and tests were chosen without risk analysis.
- Requirements did not state limits, so testers did not know the edges.
- Testers used typical input values only (the "happy path").
- Test cases were not reviewed, and the same tests were repeated (pesticide paradox).
- Exhaustive testing is impossible, and without a technique the selection of values was random.

How boundary testing helps

Boundary value analysis (BVA) is based on the observation that defects occur most frequently at the edges of input ranges, for example because of the wrong use of < instead of <=. For a range with minimum and maximum, test the values just below, on and just above each boundary. It is combined with equivalence partitioning to cover each class with a few values, giving high defect detection from few test cases.

Including edge cases in testing

Requirement	Boundary values tested
Password length 8 to 16 characters	7, 8, 9, 15, 16, 17
Withdrawal Rs.100 to Rs.20,000	99, 100, 101, 19,999, 20,000, 20,001
Quantity in cart 1 to 10	0, 1, 2, 9, 10, 11
Other edge cases	Empty value, null, maximum field length, negative numbers, special characters, leap day (29 Feb), time-zone and month-end changes, zero balance

Improvements in test design

- Define limits explicitly in requirements and review them with testers.
- Apply BVA, equivalence partitioning and robust (worst-case) boundary testing for important fields.
- Add negative and error-guessing tests and checklists of edge cases from past defects.
- Use exploratory testing and automated data generation (fuzzing, data-driven tests).
- Review and update test cases each release, and run regression tests on boundaries.

Question 19

A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts.

Answer

Introduction

Without documentation, defects are kept in e-mails and memory, so they are lost, duplicated or forgotten. A defect repository solves this by storing every defect in one structured, searchable place with its full history.

How documentation improves defect management

- **Traceability:** Every defect is linked to a requirement, test case, build and developer.
- **Tracking:** The status (new, assigned, fixed, retested, closed or reopened) is always known, and nothing is lost.
- **Communication:** Developers receive complete information, which shortens the fixing time.
- **Accountability:** Owners and dates are recorded.
- **Knowledge:** Documented defects can be reused for prevention, training and regression tests.

How structured repositories support analysis

Because defect records have common fields (module, type, severity, origin phase, root cause, status and dates), they can be filtered and counted. The team can draw Pareto charts to find defect-prone modules, trend charts for open versus closed defects, defect density by release, the

average age of defects and defect removal efficiency of each phase. These results drive decisions on release readiness, test effort and process change.

Improvements in documentation practices

- Introduce a defect tracking tool (Jira, Bugzilla, Mantis) and make it the only official place for defect reporting.
- **Use a standard template:** ID, title, description, steps, expected and actual results, environment, severity, priority, attachments, root cause and fix details.
- Define the defect life cycle and rules for updating status.
- Review for completeness and duplicates in regular triage meetings.
- Record the root cause and the fix when closing the defect.
- Maintain a regular backup, access control and periodic reports for management.

Question 20

A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach.

Answer

Redesigned approach based on testing principles

- **Early testing:** Test requirements and design of transaction rules (limits, charges, currency, rollback).
- **Risk-based testing and defect clustering:** Concentrate effort on high-risk functions (transfer, payment, standing instruction) and defect-prone modules.
- **Exhaustive testing impossible:** Select tests by equivalence partitioning, boundary values and decision tables.
- **Pesticide paradox:** Review and add test cases every release.
- **Context dependent:** Include security, integrity and audit tests because it is a financial system.
- Independent testing and a repeatable process with documentation and tools.

How structured testing improves quality

Structured testing gives planned coverage of each transaction rule, records results for repeatability and finds defects before production. Failures such as duplicate debit, lost transactions or wrong balance are found early. The defect data allow corrective action, so the failures do not repeat.

Illustration

Test case	Input / condition	Expected result
Valid transfer	Rs.5,000 with sufficient balance	Debit and credit are both completed and balance is correct
Boundary	Exactly the available balance; balance plus Rs.1	Accepted; rejected with message
Atomicity	Failure after debit, before credit	Complete rollback; no money lost
Concurrency	Two transfers from the same account at the same time	Balance consistent, no overdraft
Duplicate request	Submit button pressed twice	One transaction only
Timeout	Network drops after confirmation	Status verified, no duplicate

Techniques to enhance defect detection

- Black-box techniques (EP, BVA, decision tables, state transition) and white-box coverage of transaction code.
- Database and integration tests with the core banking and payment gateway; reconciliation tests.
- Concurrency, performance, stress and security testing.
- Automated regression integrated with continuous integration.
- Reviews, root-cause analysis and defect-repository trend analysis after each release.
- Exploratory testing by experienced testers to supplement planned tests.

Conclusion

Applying testing principles within a structured, documented and measurable process makes testing of transaction processing systematic, increases defect detection and reduces repeated failures in the banking system.